Zusicherungen, Vor- und Nachbedingungen

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Sichtbarkeit von Variablen
- Lebensdauer von Variablen
- Lokale Variablen
- Klassenvariablen

Was wird in diesem Foliensatz besprochen?

- Zusicherungen
- Vor- und Nachbedingungen
- assert-Anweisung in Java

Was lernen Sie kennen?

- Zusicherungen allgemein
- Vor- und Nachbedingungen bei Methoden
- assert-Anweisung in Java

Zusicherung (Assertion)

Ist eine **Aussage** über den **Zustand** eines Programms an einer bestimmten Stelle

Zusicherung (Beispiel)

```
if (x > y) {     // x > y
    max = x;
} else {         // !(x > y), d.h. x <= y
    max = y;
}
```

Erste Zusicherung ist trivial (kann weggelassen werden)

Die zweite Zusicherung ist in diesem Fall wichtig (<=)

Zusicherung (komplexeres Beispiel)

```
if (0 <= x && x < 10) {
    // ...
} else {
    // !(0 <= x && x < 10) wird umgeformt zu
    // !(0 <= x) || !(x < 10) oder besser
    // x < 0 || x >= 10
}
```

Umformung zusammengesetzter Ausdrücke mit Hilfe der **Gesetze von De Morgan**		
`!(a && b)`	ist äquivalent zu	`!a || !b`
`!(a || b)`	ist äquivalent zu	`!a && !b`

Zusicherungen bei Schleifen

```
i = 2;
sum = 1;
while (i <= n) {      // i <= n
    sum = sum + i;
    i = i + 1;
}
// i > n (genauer i == n + 1)
```

Summe von 1 bis n, n >= 1

Zwei einfache Zusicherungen bei while-Schleifen
• Am Anfang des Schleifenrumpfes (Schleifenbedingung) • Nach Beenden der Schleife (Negation der Schleifenbedingung)

Schleifeninvariante

```
i = 2;
sum = 1;
// sum == Summe(1 … i-1)
while (i <= n) {      // i <= n
    // sum == Summe(1 … i-1)
    sum = sum + i;
    i = i + 1;
    // sum == Summe(1 … i-1)
}
// i = n + 1 (sum == Summe(1 … i-1) == Summe (1 … n))
```

Ist eine Aussage über das Ergebnis der Schleife

Bleibt in jedem Schleifendurchlauf gleich

- Sie muss beim ersten Betreten der Schleife genauso gelten wie bei jedem weiteren Durchlauf (und auch danach)

Vorbedingungen (Preconditions)

Geben an, unter welchen Voraussetzungen das Verhalten der Methode definiert ist (z. B. korrekte Parameterbereiche)

Müssen beim Aufruf eingehalten werden

Nachbedingungen (Postconditions)

Geben an, was nach der Ausführung der Methode (direkt vor return) gelten muss

- Rückgabewerte, Ausgaben etc.

Gelten, falls Vorbedingungen eingehalten wurden

Müssen von der Methode eingehalten werden

Zusicherungen bei Methoden

Vorbedingungen

- Können bei einer öffentlichen Methode **nicht** zugesichert werden
- Zusicherung bei privaten Methoden möglich

Nachbedingungen

- Diese kann und muss die Methode garantieren
- Gelten am Ende einer Methode
- Zusicherungen zur Überprüfung verwenden

Beispiel

Vorbedingung (keine Zusicherung)
- Ist sie nicht erfüllt, dann wird kein korrektes Verhalten garantiert!

```
// a > 0 && b > 0
public static int gcd(int a, int b) {
    int first = a;
    int second = b;
    while (second != 0) {
        if (first > second) {
            first -= second;
        } else {
            second -= first;
        }
    }
    // first > 0 && a % first == 0 && b % first == 0
    return first;
}
```

Nachbedingungen (Zusicherungen)

Zusicherungen in Java

Bis jetzt nur als Kommentare geschrieben

Eigentlich sind das Aussagen, die immer wahr sein müssen

- Das könnte auch in laufenden Programmen überprüft werden

In Java gibt es dafür die assert-Anweisung

- https://docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html

assert-Anweisung in Java

Form
assert ausdruck; **assert ausdruck : ausdruck;**

```
assert a <= b && b <= c;
assert a >= 0 : "Negative a";
```

Auswertung

Der erste Ausdruck ist ein boolescher Ausdruck

- Muss zur Laufzeit true sein
- Ist er false, dann bricht das Programm ab

Der zweite Ausdruck wird ausgewertet (wenn erster Ausdruck false) und mit einer Fehlermeldung ausgegeben

```
assert a <= b && b <= c;
assert a >= 0 : "Negative a";
```

Arbeitsweise

Assertions werden zur Laufzeit ausgewertet	Wenn das Ergebnis false ist, stoppt das Programm mit einem AssertionError	Bei Abbruch wird eine Information ausgegeben
• Auswertung muss aber explizit eingeschaltet werden	• Die nachfolgenden Anweisungen werden nicht mehr ausgeführt	• Über den Ort der gescheiterten Assertion

Aktivierung

Sie können zur Laufzeit wahlweise aktiviert werden

- Sie sind automatisch deaktiviert
- Programm muss nicht neu übersetzt werden

Normalerweise während der Entwicklungszeit

- Assertions kosten Rechenzeit

Aktivierung in einem Programm Test

```
java -ea Test
```

In IntelliJ

- Unter Run->Edit Configurations
- -ea im Feld Vm Options angeben

Beispiel – Assertions für Nachbedingungen

```
public class AssertionTest {

    public static int max(int x, int y) {
        int max = 0;
        if (x < y) {
            max = x;
        } else {
            max = y;
        }
        assert max >= x && max >= y && (max == x || max == y): "Maximum is not correct";
        return max;
    }

    public static void main(String[] args) {
        System.out.println(max(10, 20));
    }
}
```

Fehler in der Implementierung!

Ausgabe ohne Überprüfung der Assertion: 10
Ausgabe mit Überprüfung:

```
Exception in thread "main" java.lang.AssertionError: Maximum is not correct
        at AssertionTest.max(AssertionTest.java:9)
        at AssertionTest.main(AssertionTest.java:15)
```

Beispiel mit mehreren Assertions

```java
public class AssertionTest2 {

    // a > 0 && b > 0
    public static int fastGcd(int a, int b) {
        int first = a, second = b;
        while (first != 0 && second != 0) {
            if (first >= second) {
                first = first % second;
            } else {
                second = second % first;
            }
        }
        int result = first + second;
        assert result > 0 : "gcd " + result + " is not > 0";
        assert a % result == 0 : "gcd " + result + " is not a divisor of " + a;
        assert b % result == 0 : "gcd " + result + " is not a divisor of " + b;
        return result;
    }

    public static void main(String[] args) {
        System.out.println(fastGcd(12, 8));
        System.out.println(fastGcd(3, 14));
    }

}
```

Schnellere Variante des Euklidischen Algorithmus